RadiusAI Outreach Generation

How research data becomes a personalised cold email — what was built, how it runs, and what stops it sending something untrue.

49	27	23	23	9	37
COMPANIES	RESEARCHED	SENDABLE	DRAFTS WRITTEN	SAFETY GATES	UNIT TESTS

1. The problem this solves

Writing one cold email by hand is easy. Writing 49 that each sound like they were written for that one institution is not — and the obvious shortcut, handing a language model a pile of notes and asking for an email, produces 49 emails that all say the same thing in different words.

> **The core rule:** raw `research_notes` prose never reaches the email-writing prompt. Free text in, generic email out. The writer only ever sees *typed fields* plus one extracted hook sentence. Everything below exists to enforce that separation.

The second problem is truth. A wrong placement percentage sent to the person whose job that number is destroys the lead permanently. So the model is never trusted to state a number — it can only repeat numbers that were extracted from a source, and code checks that afterwards.

2. How it works, end to end

```
their website
     │  fetch + summarise
     ▼
research_notes        prose. Useful to a human, dangerous to a prompt.
     │  extract()  — LLM call #1
     ▼
research_facts        typed row: 28 fields + confidence on each
     │  derive()   — pure code, no LLM
     ▼
the contract          typed facts + routing + radius_block
     │  generate() — LLM call #2
     ▼
subject + body
     │  validate() — pure code, 9 gates
     ▼
email_generations     draft (passed) or rejected (held back)
```

Two model calls, separated by deterministic code. That separation is the design. The first call only reads and types facts. The second only writes prose from facts it was handed. Neither is allowed to do both, so neither can quietly invent a fact and then write it up as true.

3. Research data — the `research_facts` table

One versioned row per institution. Re-running research keeps the old row and marks the new one current, so you can always see what was known at the time an email went out.

Tier	Fields	What it is for

Tier 0 eligibility + routing	`institution_name` , `is_valid_buyer` , `institution_type` , `campus_count` , `program_mix` , `annual_graduating_cohort` , `contact_name` , `contact_title` , `role_type` , `contact_email`	Decides *whether* to email at all and *which* template to use. If any of these is missing the send is blocked — no guessing, no "Dear Sir/Madam".
Tier 1 the hook	`recent_event` , `specificity_anchor` , `placement_cell_name` , `placement_season_window`	The line that proves this is not a blast. A verifiable detail nobody would cite about a different institution. **No hook, no send.**
Tier 2 the pitch	`claimed_placement_rate` , `median_package_lpa` , `top_recruiters` , `naac_grade` , `nirf_rank` , `tech_focus_signals` , `existing_placement_tech`	What you attach the value proposition to. Optional — an email works without these, and a wrong one is far worse than a missing one.
provenance	`field → {source_url, confidence, fetched_at}`	Every fact carries where it came from and how sure the extractor was.

The two mechanisms that make it trustworthy

- **"Output null rather than guess."** The extractor is told that a null is the *correct* answer for anything the source does not state. In practice it works: on our 27 institutions it returned null for cohort size, salary packages and placement rates rather than inventing plausible ones.
- **A 0.8 confidence floor.** Any fact scored below 0.8 is stored for audit but silently dropped before the writing prompt ever sees it. A shaky number simply does not exist as far as the email is concerned.

note Contact identity is taken from `company_contacts` , never from the model. The model only classifies the *role type* from a job title it is shown. It cannot invent a person.

4. Radius data — the `radius_block`

Everything the email says about *us*. Research data says everything about *them*. Five versions, one per segment, assembled at generation time from the pain and offer that `derive()` selected.

Part	Contents
`product_one_liner`	Input to output in one sentence.
`audience` + `posture`	Who this segment is and how much institutional weight to assume. An elite institution is not told its placements are weak.
`value_prop`	One sentence, selected by `pain_hypothesis`.
`proof`	Selected by `proof_to_cite` — only claims true today.
`offer`	The single call to action, selected by `offer_variant`.
`signature`	Read from `.env`, so a real person's credentials are never hard-coded or invented.

> **The value propositions deliberately contain no statistics.** We do not yet have a measured before/after ATS pass rate on real Indian student CVs, so any percentage here would be fabricated — exactly what the rest of the system is built to prevent. Three of the four proofs (`ats_uplift_stat`, `es_london_pilot`, `dream2rank`) are wired up but *withheld*: `PROOF_AVAILABLE` currently contains only `free_audit_only`. When you have the measured figure, add it in both places and every email starts citing it on the next run.

5. Routing — `derive()`, pure code

Runs between the two model calls. No LLM, fully deterministic, covered by unit tests. This is where personalisation actually comes from: the model never chooses the *angle*, only the prose for an angle already chosen.

Derived field	Rule
`segment_template`	Multi-campus → **group**. IIT/NIT/IIIT, central university, or NIRF ≤ 100 → **elite**. Non-engineering programme mix → **non_engineering**. Otherwise **private_uni** or **college** by type.
`pain_hypothesis`	elite → ATS rejection. Group or cohort > 2000 → staff bandwidth. Placement rate < 70 → unreported cohort. NAAC grade + published report → accreditation reporting.
`offer_variant`	Group → licence call. Everyone else → the free 50-CV audit.
`proof_to_cite`	Best available proof for the segment, filtered through `PROOF_AVAILABLE`.
`hook_sentence`	From a dated recent event, else the specificity anchor. Capped at 25 words and must clear the confidence floor.
`deal_size_band`	s / m / l / xl from cohort × campuses.

It also returns `blocked[]` — the reasons an institution must not be emailed. On the real data this caught **2 non-buyers** (a training company with no placement function, and one pharmacy institute) and **2 rows missing an institution type**.

6. Writing and checking — `generate()` then `validate()`

The writing prompt receives the contract and nothing else. What comes back is then checked by code that never asks the model whether it did well.

Gate	Rejects when
`no_orphan_numbers`	Any digit in the body does not trace to the contract or the radius block. **This is the one that stops a fabricated placement percentage.**
`confidence_floor`	A cited fact scored below 0.8.
`facts_cited_match`	The model claims a field it did not use, or names one that is not in the input.
`hook_present`	The opening line drifts more than three content words from the hook — i.e. the personalisation got written away.
`name_present`	The recipient is not addressed by name, or "Sir/Madam" appears.
`word_count`	Outside 110–140 words.
`banned_phrases`	Any em-dash, or "leverage", "cutting-edge", "revolutionise", "I hope this email finds you well", and similar.
`single_cta`	Two questions, or a link and a question together.
`dedupe`	More than 0.85 cosine similarity to the last 50 emails. This is the gate that tells you whether personalisation is *real*.

Fail closed. A blocked email costs nothing; a bad one costs the lead. A rejected email is retried *once* with its specific failures named, then re-validated from scratch — nothing is ever waved through. That retry took the pass rate from 8/24 to 23/24, because most rejections were near-misses like "107 words" or a single em-dash.

7. The audit trail — `email_generations`

One row per generated email. It stores the three things you need to explain or reproduce anything that went out:

Column	Holds
`prompt_system`, `prompt_user`, `prompt_version`	**The prompt** that was sent, verbatim.
`input_contract`	**The input used** — typed facts only, never notes.
`subject`, `body`, `facts_cited`	**The content** that came back.
`validation`, `is_valid`	Every gate's result and reason.
`segment_template`, `pain_hypothesis`, `offer_variant`, `hook_sentence`	Lifted out of the contract so you can group and filter without reading JSON.
`company_id`, `company_contact_id`, `research_facts_id`, `email_log_id`	Links back to all three source tables, and to the send once it happens.

Generating sends nothing. Rows land as `draft` or `rejected`. Sending stays a separate, explicit step.

8. What it produced on the real list

> **23 of 24 eligible contacts passed every gate, with 23 distinct subject lines.** Segments: private_uni 9 · college 8 · elite 2 · non_engineering 2 · group 2. The single rejection (BIT Mesra) rephrased its hook from "Established in 1955 by Mr. B.M. Birla" to "Since its founding by…", losing the hook words, and is correctly held back rather than sent.

A generated email, unedited

```
Dear Saloni Jain,

With over 16,000 alumni spanning across 40 countries, Poornima University
has a wide-reaching impact. However, it is common for student CVs to be
rejected by applicant tracking systems (ATS) due to formatting issues,
preventing recruiters from seeing their true potential...

Send me 50 CVs from your latest cohort and I'll provide a report on which
ones fail ATS parsing and the reasons why. Rest assured, all CVs will be
used solely for this audit and promptly deleted thereafter.
```

The only numbers in it are **16,000** and **40** — Poornima's own alumni figure, taken from their site — and **50**, which is our offer. Every one traces to the input. Anything else would have been rejected before you ever saw it.

9. Data integrity fixes

Two fields in the campaign tracker were being written by hand and had drifted out of line with reality. Both are now *derived* and cannot be hand-set:

- `research_done` — true only when a current `research_facts` row passes the Tier 0 completeness check. Notes alone no longer earn the flag; a database trigger keeps it correct.
- `campaign_status` — computed from actual `email_logs` coverage (to start / pending / completed), never asserted.

Both traced to a single demo row (C-DAC) that claimed completed status with zero emails sent. A third reported issue — `total_employees = 1` on 48 of 49 rows — turned out to be correct data rather than a bug: only one company on the list has two contacts.

10. Running it

Command	Does
`npm run companies:sync`	Rebuild companies and contacts from the Apollo list.
`npm run research:companies [n|all]`	Fetch each site and write research notes.
`npm run facts:extract [n|all] [--refresh]`	Notes → typed facts (LLM #1).
`npm run email:generate [n|all] [--groups]`	Write and validate emails (LLM #2). Sends nothing.
`npm run email:generate -- --show <id>`	Print one draft with its gates and the exact contract used.
`npm test`	37 unit tests over the routing rules and the gates.

11. Before anything is sent

1. Set the signature. `OUTREACH_SENDER_NAME`, `OUTREACH_SENDER_CREDENTIAL` and `OUTREACH_SENDER_WHY` are not set, so all 23 drafts currently end with "Best," and no name. These are claims about a real person, so they were left for you rather than invented. Set them, then regenerate.

2. Authenticate the sending domain. Delivery to Gmail still depends on SPF/DKIM/DMARC on `radiusai.online` (DNS is at one.com). Until that is done, Gmail rate-limits the shared Brevo domain and mail does not arrive.

3. Track `schema.sql` in git. It is currently in `.gitignore`, so `research_facts`, `email_generations` and every trigger exist only on this machine and in the live database.

Worth knowing

- **20 of 27 institutions have no dated recent event** and lean on a specificity anchor instead. Anchors work, but a dated event is a stronger opening — a news source would lift reply rates more than any prompt change.
- **13 companies have no website URL at all**, so they cannot be researched from their own site and need another source.
- **Only 2 multi-campus groups were found**, not the 7 assumed. Campus count is almost never stated on a homepage.
- If the `dedupe` gate ever starts firing, the fix belongs in *extraction* — thin hook data — not in the writing prompt.